
Errata: MEKF Derivation and Implementation Review

Ruixiang Du
ruixiang.du@gmail.com

ABSTRACT

This note documents the errors found in the original MEKF technical note (docs/typst/main.typ, pre-correction) and in the Mef6 implementation (src/estimation, pre-fix), explains why each is wrong and what it breaks, and maps every error to its correction and the test that now pins it. The central finding: the implementation was a **faithful implementation of a flawed derivation** — three of the five code-level defects trace directly to errors in the document.

1 Summary

Five errors were found in the derivation document and eight in the implementation. They fall into three classes: (1) **process-noise errors** in the document, copied into the code, that break the covariance's symmetry and consistency; (2) a **structural bookkeeping error** — the document omitted the bias fold-in and error-state reset, and the code consequently never accumulated bias estimates; (3) **production defects** in the code only (console I/O in the hot path, numerically fragile covariance update, no observation gating, no input validation).

2 Errors in the derivation document

2.1 D1 — Attitude-block process noise uses the wrong sigma

As printed (upper-left block of $\mathbf{Q}_d$):

$$\Lambda(\sigma_\omega^2)\Delta t + \Lambda(\sigma_\omega^2)\frac{\Delta t^3}{3} \quad (1)$$

Correct:

$$\Lambda(\sigma_\omega^2)\Delta t + \Lambda(\sigma_{\beta\omega}^2)\frac{\Delta t^3}{3} \quad (2)$$

Why: the $\Delta t^3/3$ term arises from integrating the gyro **bias random walk** through the attitude error over the step — its driver is $\sigma_{\beta\omega}$, not the gyro white noise. Evaluating $\mathbf{Q}_d = \int_0^{\Delta t} \Phi(\tau)\mathbf{Q}_c\Phi(\tau)^T d\tau$ with first-order Φ makes this explicit: the (α, α) block picks up $\sigma_\omega^2\tau^0$ and $\sigma_{\beta\omega}^2\tau^2$ integrands.

Consequence: with typical IMUs $\sigma_\omega \gg \sigma_{\beta\omega}$, so the printed formula grossly over-inflates attitude process noise at larger Δt . Note: the **code did not copy this error** — it already used $\sigma_{\beta\omega}$ here. Document-only.

2.2 D2 — The printed Q_d is asymmetric

As printed, the $(\delta v, \delta r)$ block:

$$\Lambda(\sigma_{\beta f}^2) \frac{\Delta t^4}{8} + \Lambda(\sigma_{\beta f}^2) \frac{\Delta t^2}{2} \quad (3)$$

while its mirror $(\delta r, \delta v)$ block — **correctly** — reads:

$$\Lambda(\sigma_f^2) \frac{\Delta t^2}{2} + \Lambda(\sigma_{\beta f}^2) \frac{\Delta t^4}{8} \quad (4)$$

A covariance matrix must satisfy $Q_d = Q_d^T$; the two entries disagree in which sigma multiplies $\Delta t^2/2$. The $(\delta r, \delta v)$ version is the correct one (the term couples velocity and position error through the accelerometer **white noise**).

Consequence: any faithful implementation inherits an asymmetric Q_d ; adding it every cycle destroys the symmetry and positive-definiteness of P , which is the textbook route to numerical filter divergence. **The code copied this error** (its `Q.block(3,6)` used $\sigma_{\beta f}$ where its own `Q.block(6,3)` used σ_f).

2.3 D3 — Gyro-bias random walk grows as the wrong power of Δt

As printed: $Q_d^{(\beta\omega, \beta\omega)} = \Lambda(\sigma_{\beta\omega}^2) \Delta \frac{t^2}{2}$. Correct: $\Lambda(\sigma_{\beta\omega}^2) \Delta t$.

Why: a random-walk state's variance grows linearly in time — and the **same matrix** printed the accelerometer-bias twin correctly as $\Lambda(\sigma_{\beta f}^2) \Delta t$. The two entries are structurally identical; one of them had a typo.

Consequence: the filter under-weights gyro-bias uncertainty growth (for $\Delta t < 2$ s), slowing or stalling bias convergence. **The code copied this error.**

2.4 D4 — The measurement update never uses the innovation

As printed:

$$\delta \hat{x}_k^+ = \delta \hat{x}_k^- + K_k H_k \delta x_k^- \quad (5)$$

As written, the measurement δz_k **does not appear** — the update multiplies the gain by the predicted error state itself, which is identically zero after a reset. The update law is non-functional as printed.

Correct (the innovation form):

$$\delta \hat{x}_k^+ = \delta \hat{x}_k^- + K_k (\delta z_k - H_k \delta \hat{x}_k^-) \quad (6)$$

which, because the error state is reset after every fold, reduces to $\delta \hat{x}_k^+ = K_k \delta z_k$. (The code implemented the correct form — this was a document-only transcription error, but a reader implementing from the document would produce a controller that never corrects.)

2.5 D5 — The full-state fold omitted the biases and the reset

As printed, the “update the full states” step folded only:

$$\hat{q}_k^+, \quad \hat{r}_k^+, \quad \hat{v}_k^+ \quad (7)$$

Missing: the bias fold $\hat{\beta}^+ = \hat{\beta}^- + \beta^+$ for all three bias groups, and the explicit error-state reset $\delta \hat{x}_k^+ \leftarrow 0$.

Why it matters: in an error-state (indirect) formulation, the filter estimates **errors**; every quantity the measurement update produces must be folded into a persistent nominal state and

the error zeroed, or the information is lost on the next cycle. Omitting the bias fold makes the bias states decorative: $\mathbf{P}$ converges as if biases were being learned while no bias estimate ever accumulates. **This is the direct root cause of the worst implementation bug (C1).**

3 Errors in the implementation (pre-fix mekf6.cpp)

3.1 C1 — Bias states never persisted (structural)

The old update cycle was:

```
// line 79-80: subtract "bias" from measurements
ControlInput gyro = gyro_tilde - x_.segment<3>(9);
Observation accel = accel_tilde - x_.segment<3>(12);
// line 83: reset the error state
x_ = State::Zero();
...
// line 146: measurement update
x_ = x_ + K * delta_y;
// lines 159-160: "fold"???
x_.block<3,1>(0,0) += x_.block<3,1>(9,0);
x_.block<3,1>(3,0) += x_.block<3,1>(12,0);
```

Three things are wrong at once:

1. The “bias” subtracted at the top is whatever the **previous single** measurement update deposited in slots 9–14 — not an accumulated estimate. Because $\mathbf{x}_-$ is zeroed each cycle (line 83), the bias estimate restarts from zero every step. Bias can never converge, while $\mathbf{P}_-$ (updated by the full Kalman recursion) **claims** it has — the filter is statistically inconsistent with itself.
2. Lines 159–160 add the bias error into the attitude/velocity error slots (rows 0–2, 3–5) — slots that nothing reads before the next reset. Dead writes masquerading as the fold step.
3. There were no nominal bias members at all — nowhere for an accumulated estimate to live. This is D5 faithfully implemented.

Fix: nominal members $\mathbf{b}_{\text{omega}}$, $\mathbf{b}_f$; measurements corrected against them; after each update $\mathbf{b}_{\text{omega}} += \mathbf{x}.\text{segment}<3>(9)$, $\mathbf{b}_f += \mathbf{x}.\text{segment}<3>(12)$, quaternion fold, error reset. **Pinned by:** `Mekf6Test.GyroBiasRecovery` — a constant injected gyro bias must be recovered to 20% on the gravity-observable axes; the old code cannot pass it structurally.

3.2 C2 / C3 — Process-noise errors inherited from D2 / D3

`GetQMatrix()` reproduced the asymmetric $(\delta v, \delta r)$ entry and the $\Delta t^2/2$ gyro-bias walk. **Fix:** corrected terms; the matrix is now symmetric **by construction** (each lower block assigned as the transpose of its upper mirror). **Pinned by:** `Mekf6Test.CovarianceStaysSymmetricPositive` — 5000 cycles, $\|\mathbf{P} - \mathbf{P}^T\| < 10^{-12}$ and smallest eigenvalue $> -10^{-12}$.

3.3 C4 — Dead code revealing the reset confusion

$\mathbf{x}_- = \Phi * \mathbf{x}_-$; immediately after $\mathbf{x}_-$ was zeroed — always the zero vector. Harmless numerically; symptomatic of the misplaced reset. Removed (the error-state mean is zero by construction after every fold; only $\mathbf{P}$ propagates).

3.4 C5 — Console I/O in the filter hot path

Four unconditional `std::cout` statements ran on **every** `Update()` — at a 100 Hz IMU rate, formatted console output inside the control path (blocking, allocating, non-deterministic). Removed; diagnostics now flow through the wait-free telemetry API and accessors.

3.5 C6 — Numerically fragile covariance update

$P \leftarrow (I - KH)P$ is the textbook-but-unstable form: it is exact only for the exactly-optimal K and loses symmetry under floating point. **Fix:** Joseph form $(I - KH)P(I - KH)^T + K R K^T$ plus explicit re-symmetrization, and an LDLT solve instead of forming the innovation-matrix inverse.

3.6 C7 — No accelerometer gating

The gravity observation was applied unconditionally. Under dynamic acceleration the accelerometer does **not** measure gravity, and the update drags the attitude toward a false vertical. **Fix:** gate on $||\mathbf{a}|| - g$ (configurable, default 0.5 m/s²); rejected cycles are prediction-only. **Pinned by:** `Mekf6Test.AccelGateRejectsDynamicAcceleration` — 5 m/s² lateral acceleration for 200 cycles moves the attitude estimate by less than 0.1°.

3.7 C8 — Minor defects

Float literals (2.0f, 3.0f) mixed into double expressions; `q.inverse()` where `conjugate()` suffices on a normalized quaternion; no input validation (`dt <= 0`, NaN propagate silently — now rejected with the state untouched, pinned by `Mekf6Test.RejectsInvalidInput`).

4 Error-to-fix map

Error	Corrected in	Pinned by
D1 sigma in $Q^{\alpha\alpha}$	document	—(doc-only; code was right)
D2 / C2 asymmetric Q_d	document + code	<code>CovarianceStaysSymmetricPositive</code>
D3 / C3 bias-walk power	document + code	<code>GyroBiasRecovery</code> (convergence rate)
D4 update law	document	—(doc-only; code was right)
D5 / C1 missing bias fold + reset	document + code	<code>GyroBiasRecovery</code>
C4 dead propagation	code	—(removed)
C5 console I/O	code	—(removed; telemetry instead)
C6 covariance update form	code	<code>CovarianceStaysSymmetricPositive</code>
C7 no gating	code	<code>AccelGateRejectsDynamicAcceleration</code>
C8 validation/hygiene	code	<code>RejectsInvalidInput</code>

Behavioral correctness of the whole is additionally pinned by `StaticAttitudeConvergence` (12° initial error converging below 0.5°, with the attitude/accel-bias observability manifold documented in the test) and `TracksSlowRollRotation` (sustained-motion tracking, gravity-direction error below 1°).

5 A process observation

The implementation was not careless — it was **faithful to a flawed specification**. Three of the five substantive code defects (C1, C2, C3) are exact transcriptions of document errors (D5, D2, D3). The corrective lesson applied here: fix the derivation first, then implement against the corrected document, and let an independent oracle (the discrete LQR analogue for the linear case; the synthetic-truth suite here) arbitrate between them. The reference lineage now cited in the main note (Lefferts–Markley–Shuster 1982; Markley 2003; Trawny–Roumeliotis 2005; Maley 2013; Solà 2017) provides the third-party ground truth both artifacts are checked against.